Actividad – Git

1. Git – Control de versiones

Git es un sistema de control de versiones. El objetivo de control de versiones es, sobre una serie de archivos (de código, imágenes, texto, etc), controlar quién los guardó, cuándo los guardó, por qué los guardó y cuáles fueron los cambios que sufrieron dichos archivos. Además, en el caso de estar trabajando en un equipo de desarrolladores, el control de versiones permite que cada uno se centre en una funcionalidad diferente (trabajando en archivos diferentes o incluso en los mismos archivos) y que finalmente el trabajo de todos se fusione en una versión estable. Esto normalmente se hace usando repositorios alojados en una plataforma web, como por ejemplo GitHub, que nos permite trabajar de forma remota.

Git controla los cambios de nuestro código de forma local y además nos permite usar repositorios alojados en Internet.

GitHub es una plataforma web que nos permite alojar repositorios y subir diferentes versiones de nuestro código con Git.

En este actividad vamos a:
- crear una cuenta en GitHub
- instalar Git en nuestra máquina local.
- subir a un repositorio en GitHub un proyecto que ya tengamos creado.
- trabajar desde cero con un repositorio en GitHub.
- subir cambios a nuestro repositorio.

2. Crear una cuenta en GitHub

Para usar **GitHub** primero debemos **crearnos una cuenta**:

https://github.com/ y picamos en **Sign Up**

Nos pedirá un email y un nombre de usuario. Tanto el email como el nombre de usuario los usaremos más tarde para configurar el Git en nuestro equipo.

3. Instalar Git en nuestra máquina local

Git es el software que nos va a controlar los cambios de nuestro código de forma local. Cuando lo vinculemos a nuestra cuenta de GitHub podremos subir esos cambios a la plataforma GitHub.

Para descargar git nos vamos a su página oficial:

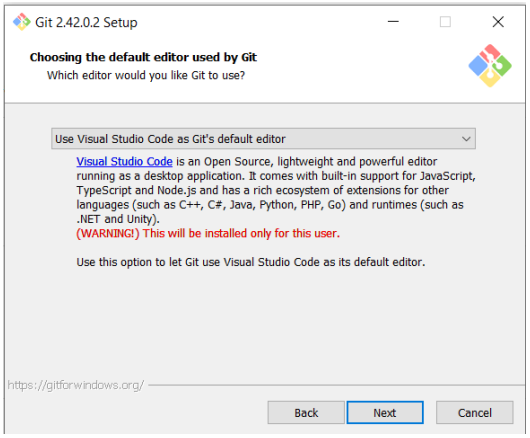
https://git-scm.com/

Ahí picamos en *Donwloads,* buscamos nuestro sistema operativo (Windows) y picamos en *Click here to download* (la última versión de Git para Windows en este momento es la 2.46.1).

Actividad – Git

Se les descargará un ejecutable.

En el proceso de instalación elegir el visual studio code como el editor por defecto del Git:



En las demás ventanas del instalador dejamos todas las opciones tal y como vienen.

4. Configurar Git

En cualquier terminal, puede ser la del Visual Studio Code o el propio cmd de Windows comprobamos primero la versión de git instalada:

```
C:\Users\Patricia>git -v
git version 2.46.1.windows.1
```

Ahora vamos a configurar el usuario y el email que usará el git. Puesto que lo vamos a vincular a GitHub pondremos el mismo nombre de usuario e email que pusimos en nuestra cuenta de GitHub.

Configuramos el usuario en la terminal:

git config --global user.name “username”

```
git config --global user.name "nombreusuario"
```

Configuramos el email en la terminal:

git config --global user.email “email”

```
git config --global user.email "emailusuario@gmail.com"
```

Para comprobar qué user.name y user.email pusiste los comandos son:
git config user.name
git config user.email

Actividad – Git

5. Subir un proyecto ya creado a GitHub usando Git

5.1. Inicializar git en mi proyecto, añadirlo a zona de espera y hacer commit

Entramos en la carpeta de nuestro proyecto (ustedes háganlo con el proyecto *mianimal*):

cd miproyecto

Escribimos el comando:

git init

El comando **git init** inicializa las estructuras de control de versiones de Git en nuestro proyecto. Si navegan con el explorador de archivos a la carpeta de su proyecto verán que ha aparecido una carpeta oculta llamada .git. Ahí están las estructuras de control de Git y además se almacenarán los snapshots que se hagan de los ficheros a lo largo del tiempo.



Ahora escribimos en la línea de comandos:

git add .

El comando git add . comprueba qué ficheros han cambiado y los pone en una especie de zona de espera esperando a que se realice un commit.
También podríamos poner en la zona de espera un fichero en concreto: git add <ruta del fichero>

Ahora hacemos un commit:

git commit -m "Mi primer commit: primera versión"

Un *commit* realiza el snapshot (una foto) del estado actual de los ficheros que están en la zona de espera. El parámetro -m sirve para describir de qué estamos "sacando la foto". Es altamente recomendable describir qué estamos haciendo en un *commit* determinado porque nos ayuda a seguir el control de los cambios.

Si ponemos: **git log --pretty=oneline** podemos ver un log de los commit.

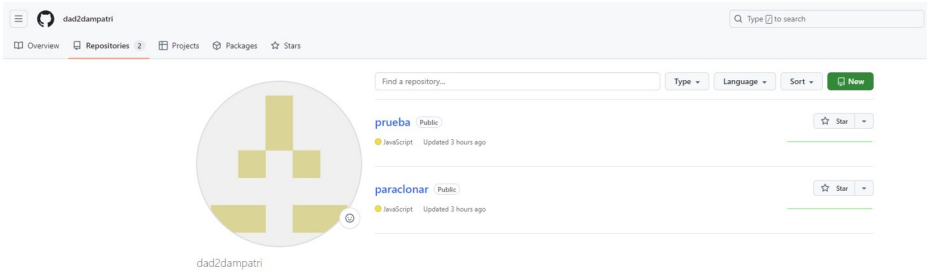
Hasta ahora todo esto ha sido local. Si yo hago un cambio en otro fichero, puedo volver a repetir los comandos git add y git commit para hacer otra foto del estado actual de mis ficheros, pero a nivel local.

Ahora vamos a subir el commit a GitHub. Para ello en GitHub creamos un repositorio y le ponemos el mismo nombre que tiene nuestro proyecto en local (*pruebas* en el caso de ustedes).

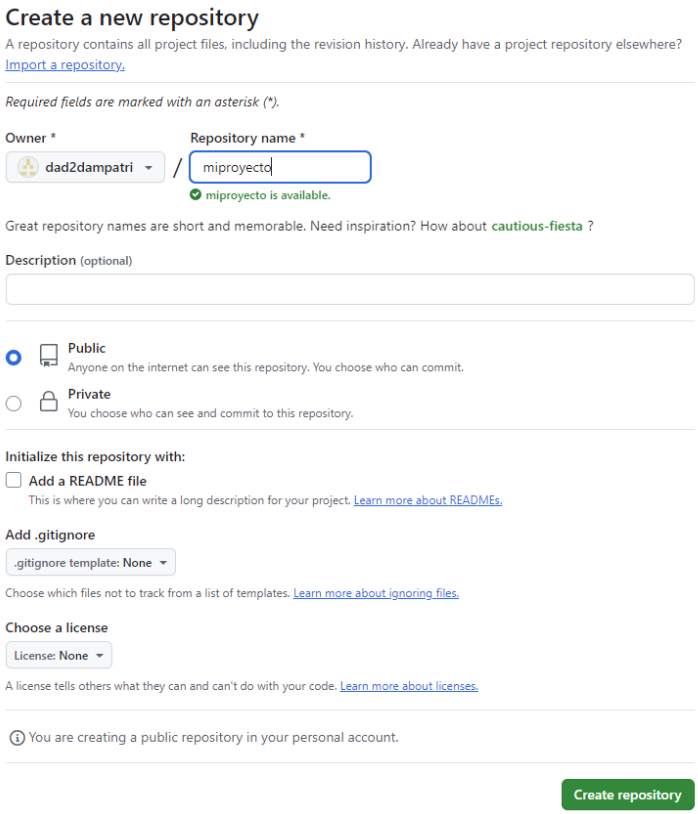
Actividad – Git

5.2. Crear un repositorio en GitHub con el mismo nombre que mi proyecto

Entro en mi cuenta de GitHub y creo un nuevo repositorio: entra en Repositories y luego pica en *New*. Si es la primera vez que entras a GitHub probablemente entres directamente a una ventana en la que te preguntarán el nombre del nuevo repositorio.



En la siguiente página tienen que rellenar el ***Repository name*** con el nombre de su proyecto (*mianimal*) y deben seleccionar que el repositorio sea ***Public***. Abajo del todo picas en ***Create repository***



5.3. Subir commits al repositorio de GitHub

Ahora volvemos a nuestra terminal en nuestra máquina local para subir nuestros commits a nuestro repositorio en GitHub.

Cambiamos primero el nombre de la rama master a main:

git branch -M main

Creamos un repositorio remoto al que llamaremos origin (es el nombre por defecto que le da Git) y al cual asociaremos nuestro repositorio de GitHub

git remote add origin https://github.com/dad2dampatri/miproyecto.git

Ahora subo al repositorio origin (https://github.com/dad2dampatri/miproyecto.git) los commits de antes con el siguiente comando:

git push -u origin main

Estoy subiendo el commit al repositorio origin (https://github.com/dad2dampatri/miproyecto.git) a la rama main: git push -u <repositorio> <rama>

En resumen, suponiendo que hayamos creado ya el repositorio en GitHub, tendríamos que realizar estos pasos en la máquina local:

cd miproyecto
git init
git add .
git commit -m “Hago un commit”
git branch -M main
git remote add origin <https://github.com/NOMBREUSUARIO/NOMBREPROYECTO.git>
git push -u origin main

5.4 Una vez inicializado y subido mi primer commit, ¿qué comandos uso cuando quiero hacer más commits?

Si ahora lo que quiero es seguir trabajando en mi proyecto local y hacer un cambio y subirlo, los únicos comandos que tengo que usar son:

git add .
git commit -m “Botón hecho”
git push

6. Subir un proyecto a GitHub cuando el proyecto no ha sido creado en local

Este es el caso en el que aún no hemos creado ningún proyecto con React+Vite y ya tengo claro que voy a trabajar con GitHub. La forma de proceder en este caso es:

6.1 Crear un repositorio en el GitHub

Creamos un repositorio en GitHub con el nombre que queramos ponerle a nuestro proyecto. Esta parte ya la sabes hacer.

Actividad – Git

6.2 Clonar el repositorio en local

En una terminal te sitúas en la carpeta contenedora de todos tus proyectos React (ProyectosReact) y ejecutas el comando:

git clone <url de tu repositorio en github>

Se creará una carpeta con el nombre de tu proyecto ya con una carpeta oculta .git para el control de versiones.

6.3 Crear proyecto React + Vite en nuestro equipo

Nos situamos en la carpeta contenedora de nuestros proyectos React, y en la terminal del Visual Studio Code creamos un proyecto React + Vite como ya sabemos: npm create vite@latest

Le ponemos el mismo nombre que la carpeta ya creada.

6.4 Subir a repositorio remoto (GitHub)

Puesto que hemos hecho un clone, ya está creada la conexión entre mi proyecto local y el repositorio en GitHub, así que lo único que tenemos que hacer para subir cambios es lo de siempre:

git add .
git commit -m “otro commit”
git push

7. Fichero .gitignore

Recordemos que cuando creamos un proyecto React+Vite se crea un fichero llamado .gitignore. Este fichero indica a git qué ficheros NO se van a incluir en nuestro repositorio. Quiero que se fijen expresamente en la línea 10, donde vemos la carpeta node_modules. Esta carpeta no se incluye puesto que pesa mucho. Contiene como saben las dependencias/librerías ya instaladas en nuestro proyecto.

